Senior Backend Architecture Thinking

From Tool-First Memorization to Problem-First Engineering

THE SENIOR MINDSET SHIFT

The Beginner Path

Focuses on **tools first**. Learning gRPC, Kafka, or GraphQL without knowing the underlying business pain.

"What is Kubernetes?"

The Senior Path

Focuses on **trade-offs**. Prioritizing operational cost, team velocity, and user impact over technology hype.

"What costs us money?"

Monolith vs. Microservices

The Evolution of Organizational Scaling

WHY MONOLITHS ARE CORRECT

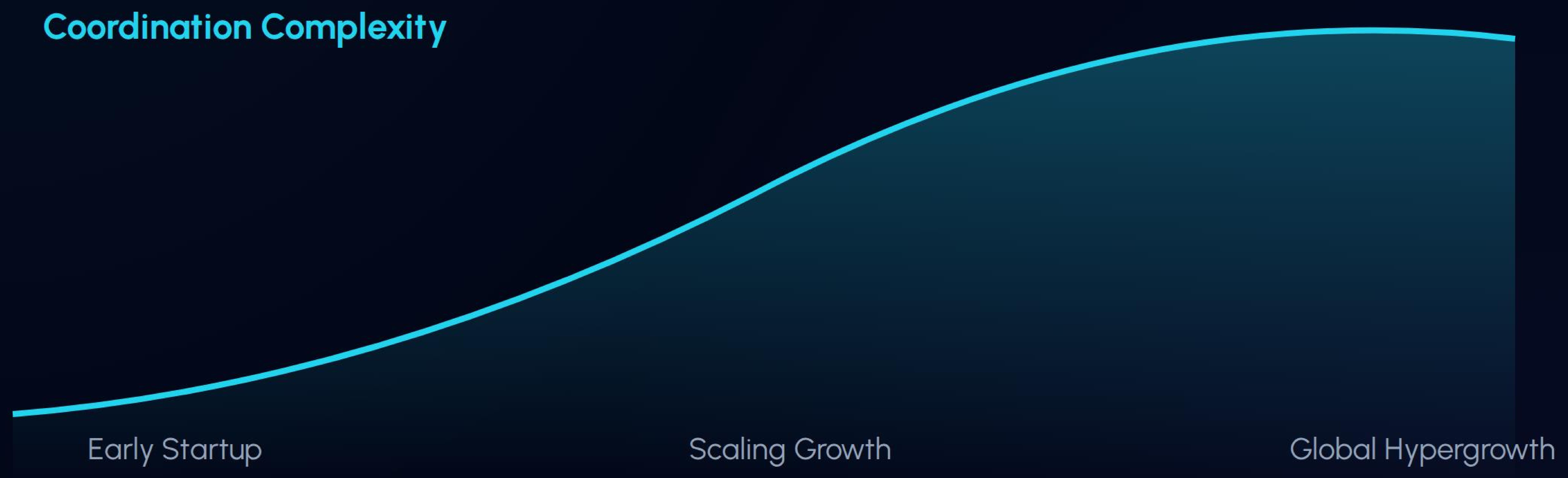
Startups prioritize **speed and simplicity**. For a team of 2-5 developers, a monolith is usually the superior choice.

- **Low Latency:** Function calls are instant.
- **Easy Debugging:** Unified logs and state.
- **Cheap:** Single server, single DB.

Organized high-end kitchen

WHEN MONOLITH PAIN PEAKS

As Uber grew from 10 to 1000 engineers, the **coordination cost** of a single codebase became a bottleneck.



SPLITTING BY BUSINESS DOMAIN

Rider Domain

Handles profiles, loyalty programs, and accounts.

Trip Domain

Manages the lifecycle of a ride from request to completion.

Payment Domain

Processes transactions, refunds, and fraud checks.

INTER-SERVICE COMMUNICATION

THE API GATEWAY PATTERN

Responsibility	Senior Justification
Authentication	Centralized logic prevents security fragmentation across services.
Rate Limiting	Protects downstream infrastructure from abuse or spikes.
Protocol Translation	Exposes REST to clients while using gRPC internally for speed.
Observability	Primary entry point for request tracing and latency monitoring.

DATABASE PER SERVICE PATTERN

- **Auth DB:** SQL (Relational Users)
- **Matching DB:** Redis (High-speed Geospatial)
- **Analytics DB:** ClickHouse (OLAP/Logs)

Decoupling storage allows teams to choose the **correct tool** for their specific domain needs.

REAL-WORLD OBSERVABILITY

In a distributed system, tracing is not optional. Senior engineers follow a single request's journey across 20+ services.

- **Tracing:** Tracking Span IDs across services.
- **Monitoring:** Health of CPU, Memory, and Disk.
- **Logging:** Structured events for post-mortems.

THE ESTIMATION FRAMEWORK

Before picking a tool, ask these 6 critical questions:

- **Scale:** How many concurrent users at peak?
- **Latency:** Is this a real-time requirement or async?
- **Tolerance:** What happens if this service fails?
- **Consistency:** Do we need ACID or Eventual Consistency?
- **Complexity:** Does the team have the expertise for this tool?

Final Industry Truth

"Engineering is the art of choosing the **least-bad trade-off** for your current constraints."

Seniority = Judgement.

Tools change. Problems remain.

IMAGE SOURCES

Thumbnail for triline.co.uk

https://triline.co.uk/wp-content/uploads/2024/04/Asset-4.png

Source: triline.co.uk

Thumbnail for dribbble.com

https://cdn.dribbble.com/userupload/47648494/file/6a805a6bb67f3517b3ac8c71ce2b6c22.jpg?format=webp&resize=400x300&vertical=center

Source: dribbble.com